

Latent variable Models : AE and VAE

Nagesh

May 2026

A word about data compression :

Data compression(encoding) is what we store as stories, memories or encoded information in our brains Stories are compressed data of some timely experience in brain.

This is the motivation behind compression(Encoding) and expansion(Decoding) of the data.

We don't learn everything, we learn a general distribution of behaviour, language, stories and so on . According to this distribution we sample action based on situations (intelligence is emergent property of humans, not implicit one like heart beat) If machine does that , we call it AI The world is a representation learning" // (Nagesh)

Contents

Introduction to Latent Variable Methods	8
Definition of a Latent Variable Model	8
What is a Latent Variable?	9
Interpretation of z	9
Mathematical Perspective	9
Latent Variable Associated with Each Data Point	10
Example A : Discrete Latent Variable	10
Example : Image Clustering	11
Examples of Discrete Latent Variable Methods	11
Example B : Continuous Latent Variable	11
Example : Autoencoder	12

Why Continuous Latent Variables are Useful	12
Important Note	13
Evidence Lower Bound (ELBO)	13
Goal	14
Learning via KL Divergence Minimization	14
Entropy Term	14
Maximum Likelihood Estimation (MLE)	15
Difficulty in Latent Variable Models	15
Introducing a Variational Distribution	15
Jensen's Inequality	16
Evidence Lower Bound (ELBO)	16
Interpretation of ELBO	17
Optimization Problem	17
Variational Posterior	17
Why ELBO is Important	18
Expectation Maximization (EM)	18
Important Notes	18
Key Takeaway	19
Example Case : Gaussian Mixture Model (GMM)	19
Latent Variable Representation	19
Gaussian Mixture Assumption	20
Interpretation	20
Parameters of a GMM	20
Constraints on Mixing Coefficients	21
Goal	21

Expectation Maximization (EM) Algorithm	21
Likelihood Improvement Property of EM	22
E-Step for GMM	22
Posterior Computation Using Bayes Theorem	23
Interpretation of Responsibilities	23
M-Step for GMM	23
Closed Form EM Updates	24
Relation Between K-Means and GMM	24
Key Difference Between GMM and K-Means	25
Important Note About EM	25
When EM Fails	25
Neural Latent Variable Models	26
Goals of Neural Latent Variable Models	26
Latent Variable Model	26
Expanding the ELBO	27
Interpretation of ELBO Terms	27
Loss Function for Neural Latent Variable Models	28
How Neural Networks Represent Distributions	28
A. Deterministic Representation	28
B. Probabilistic Representation	29
Autoencoders (AE) and Variational Autoencoders (VAE)	29
VAE Architecture	29
Encoder-Decoder Structure	30
Important Note	30
Optimization	30

Autoencoder (AE) as a Special Case of VAE	31
Autoencoder Objective	31
VAE Objective	31
Limiting Case	31
Key Difference Between AE and VAE	32
Key Takeaway	32
Training VAE : Reparameterization Trick	32
Data Flow in Autoencoder (AE)	32
Data Flow in Variational Autoencoder (VAE)	33
Important Difference	33
VAE Objective	34
Problem : Computing Gradients Through Sampling	34
Problem	35
Important Conclusion	35
Solution : Reparameterization Trick	35
Core Idea	35
Key Advantage	36
Monte Carlo Approximation	36
Gaussian Reparameterization	37
Interpretation	37
Affine Transformation Interpretation	37
Important Observation	38
Inverse CDF Reparameterization	38
Examples	38
Why Reparameterization is Important	39

Key Takeaway	39
Final Training of Variational Autoencoder (VAE)	39
Forward Computational Graph	39
Sampling via Reparameterization	40
Decoder	40
Reconstruction Term in ELBO	41
Likelihood Modeling	41
Gaussian Likelihood	42
Reconstruction Loss	42
Forward Pass of VAE	42
Backward Pass	43
Second Term in ELBO : KL Divergence	44
Closed Form KL Divergence	44
Total Gradient for Encoder	44
Training the Decoder	45
Summary of VAE Training	45
Final VAE Loss	45
Key Takeaway	46
Inference with Variational Autoencoders (VAE)	46
(i) Data Generation or Sampling	46
Sampling Procedure	46
Decoder Output	47
Why Sampling from $\mathcal{N}(0, I)$ Works	47
Important Interpretation	48
Two Ways to Generate Data	48

Important Note	48
Latent Space Interpretation	49
(ii) VAE for Posterior Inference / Embedding / Feature Extraction	49
Encoder Inference	49
Latent Representation	49
Dimensionality Reduction	50
Applications of VAE Embeddings	50
Latent Space Geometry	51
Comparison : AE vs VAE Latent Space	51
Key Takeaway	51
β -Variational Autoencoder (β -VAE)	52
Posterior Collapse in Naive VAE	52
What Does the KL Term Do?	52
Posterior Collapse	52
Problem	53
Definition : Posterior Collapse	53
Effect of Posterior Collapse	53
Understanding VAE as a Regularized Autoencoder	53
What is Regularization?	54
Effect of Regularization Constant	54
Observation About Naive VAE	55
Solution : β -VAE	55
Interpretation of β	55
Small β	55

Large β	56
Trade-off in β-VAE	56
Key Insight	57
Latent Space Effect	57
Disentangled Representation Learning	57
Final Objective of β-VAE	57
Key Takeaway	58

Introduction to Latent Variable Methods

Suppose our dataset is:

$$D = \{x_1, x_2, \dots, x_n\}$$

where:

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

Assume:

$$P_\theta$$

denotes a parametric probabilistic model.

Definition of a Latent Variable Model

A latent variable model introduces an additional hidden random variable:

$$z$$

which is not directly observed in the dataset.

The probability model is defined as:

Discrete Latent Variable

If:

$$z$$

is discrete:

$$P_\theta(x) = \sum_z P_\theta(x, z)$$

This is called marginalization over the latent variable.

Continuous Latent Variable

If:

$$z$$

is continuous:

$$P_\theta(x) = \int P_\theta(x, z) dz$$

Thus:

$$z$$

is integrated out to obtain the marginal probability of:

$$x$$

What is a Latent Variable?

The variable:

z

is called:

- Latent variable
- Hidden variable
- Unobserved variable

because it is not directly observed in the dataset.

Interpretation of z

What:

z

represents depends on the application.

For example, in image datasets:

z

may encode:

- Orientation
- Rotation
- Object size
- Illumination
- Pose
- Semantic content

Mathematical Perspective

From a mathematical viewpoint:

z

is a synthetically introduced hidden random variable that captures underlying structure or information about the data.

Although:

z

is not directly observed during data collection, it is assumed to explain variations present in the data.

Typically:

$$z$$

is jointly estimated along with the model parameters:

$$\theta$$

Latent Variable Associated with Each Data Point

For every sample:

$$x_i \in D$$

we assume there exists a corresponding latent variable:

$$z_i$$

Thus:

$$(x_i, z_i)$$

together describe the complete probabilistic model.

Example A : Discrete Latent Variable

Suppose:

$$z \in \{1, 2, 3, \dots, M\}$$

and:

$$x \in \mathbb{R}^d$$

Then:

$$z_i | x_i$$

represents the latent category associated with:

$$x_i$$

Since:

$$z_i$$

is discrete, each data sample belongs to one of:

$$M$$

categories or clusters.

Example : Image Clustering

Suppose:

$$x_i$$

is a handwritten digit image of size:

$$28 \times 28$$

A latent variable model may map the image into:

$$(28, 3)$$

meaning the image belongs probabilistically to one of three clusters.
For example:

- Cluster 1 : Digits with circular shapes
- Cluster 2 : Digits with vertical strokes
- Cluster 3 : Digits with diagonal structures

Examples of Discrete Latent Variable Methods

- Gaussian Mixture Models (GMM)
- K-Means Clustering
- DBSCAN
- Hidden Markov Models

Example B : Continuous Latent Variable

Suppose:

$$x \in \mathbb{R}^d$$

and:

$$z \in \mathbb{R}^k$$

with:

$$k \ll d$$

Then:

$$z$$

is a low-dimensional continuous representation of:

$$x$$

Example : Autoencoder

In autoencoders:

$$z_i | x_i$$

represents a compressed feature representation of:

$$x_i$$

For example:

$$x_i \in \mathbb{R}^{784}$$

may be compressed into:

$$z_i \in \mathbb{R}^{16}$$

where:

$$z_i$$

captures important semantic features of the input.

Why Continuous Latent Variables are Useful

Continuous latent spaces can learn:

- Smooth feature representations
- Semantic structure
- Data manifolds
- Low-dimensional embeddings

They are widely used in:

- Autoencoders
- Variational Autoencoders (VAE)
- Diffusion Models
- Representation Learning

Important Note

Latent variable models are not only useful for representation learning.
They can also be used as:

- Generative models
- Density estimation models
- Clustering methods
- Compression models

By sampling latent variables:

$$z$$

and generating:

$$x$$

from:

$$P_{\theta}(x|z)$$

one can generate entirely new synthetic data samples.

Evidence Lower Bound (ELBO)

Let us now understand the general principle behind learning latent variable models.

Suppose we are given a dataset:

$$D = \{x_i\}_{i=1}^n$$

where:

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

Assume the probabilistic latent variable model:

$$P_{\theta}(x) = \int P_{\theta}(x, z) dz$$

where:

$$z$$

is a latent variable.

Goal

Our goal is to estimate the model parameters:

$$\theta$$

such that:

$$P_\theta(x)$$

approximates the true data distribution:

$$P_X(x)$$

Learning via KL Divergence Minimization

We define the optimization problem as:

$$\theta^* = \arg \min_{\theta} D_{KL}(P_X \parallel P_\theta)$$

Expanding the KL divergence:

$$\theta^* = \arg \min_{\theta} \int P_X(x) \log \frac{P_X(x)}{P_\theta(x)} dx$$

Separating the terms:

$$= \arg \min_{\theta} \left[\int P_X(x) \log P_X(x) dx - \int P_X(x) \log P_\theta(x) dx \right]$$

Entropy Term

The first term:

$$\int P_X(x) \log P_X(x) dx$$

depends only on the true data distribution:

$$P_X$$

and is independent of:

$$\theta$$

Therefore it can be removed from optimization.

Thus:

$$\theta^* = \arg \min_{\theta} \left[- \int P_X(x) \log P_\theta(x) dx \right]$$

or equivalently:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{P_X} [\log P_\theta(x)]$$

Maximum Likelihood Estimation (MLE)

The quantity:

$$\log P_{\theta}(x)$$

is called the **log-likelihood**.

Thus the optimization problem becomes:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{P_X} [\log P_{\theta}(x)]$$

This is known as:

Maximum Likelihood Estimation (MLE)

Difficulty in Latent Variable Models

For latent variable models:

$$P_{\theta}(x) = \int P_{\theta}(x, z) dz$$

Therefore:

$$\log P_{\theta}(x) = \log \int P_{\theta}(x, z) dz$$

The logarithm is outside the integral.

For complex models, this integral is often intractable.

Introducing a Variational Distribution

Suppose:

$$q(z|x)$$

is any probability distribution over:

$$z$$

called the **variational posterior**.

Multiply and divide inside the integral by:

$$q(z|x)$$

$$\log P_{\theta}(x) = \log \int P_{\theta}(x, z) \frac{q(z|x)}{q(z|x)} dz$$

Rearranging:

$$= \log \int q(z|x) \frac{P_\theta(x, z)}{q(z|x)} dz$$

Recognizing expectation:

$$= \log \mathbb{E}_{q(z|x)} \left[\frac{P_\theta(x, z)}{q(z|x)} \right]$$

since:

$$\int q(z|x) g(z) dz = \mathbb{E}_{q(z|x)}[g(z)]$$

Jensen's Inequality

The logarithm function is concave.

For concave functions:

$$\log(\mathbb{E}[X]) \geq \mathbb{E}[\log X]$$

Applying Jensen's inequality:

$$\log \mathbb{E}_{q(z|x)} \left[\frac{P_\theta(x, z)}{q(z|x)} \right] \geq \mathbb{E}_{q(z|x)} \left[\log \frac{P_\theta(x, z)}{q(z|x)} \right]$$

Thus:

$$\log P_\theta(x) \geq \mathbb{E}_{q(z|x)} \left[\log \frac{P_\theta(x, z)}{q(z|x)} \right]$$

Evidence Lower Bound (ELBO)

Define:

$$\ell(\theta) = \log P_\theta(x)$$

called the **evidence** or **log marginal likelihood**.

Define:

$$J_\theta(q) = \mathbb{E}_{q(z|x)} \left[\log \frac{P_\theta(x, z)}{q(z|x)} \right]$$

Then:

$$\ell(\theta) \geq J_\theta(q)$$

Thus:

$$J_\theta(q)$$

is called the:

Evidence Lower Bound (ELBO)

Interpretation of ELBO

The ELBO is a lower bound on the true log-likelihood:

$$\log P_{\theta}(x)$$

Instead of optimizing the intractable likelihood directly, we optimize the ELBO.

Optimization Problem

Original optimization:

$$\theta^* = \arg \max_{\theta} \ell(\theta)$$

Approximate optimization using ELBO:

$$\theta^*, q^* = \arg \max_{\theta, q} J_{\theta}(q)$$

Expanding:

$$\theta^*, q^* = \arg \max_{\theta, q} \mathbb{E}_{q(z|x)} \left[\log \frac{P_{\theta}(x, z)}{q(z|x)} \right]$$

Variational Posterior

The distribution:

$$q(z|x)$$

is called the:

Variational Latent Posterior

It approximates the true posterior:

$$P_{\theta}(z|x)$$

which is often computationally intractable.

Why ELBO is Important

ELBO transforms an intractable latent variable optimization problem into a tractable expectation maximization problem.

This forms the mathematical foundation of:

- Variational Autoencoders (VAE)
- Diffusion Models
- Probabilistic Graphical Models
- Bayesian Deep Learning
- Expectation Maximization (EM)

Expectation Maximization (EM)

The optimization:

$$\theta^*, q^* = \arg \max_{\theta, q} J_{\theta}(q)$$

is often solved using the:

Expectation Maximization (EM) Algorithm

Important Notes

1. ELBO Optimization Guarantees

Optimizing:

$$J_{\theta}(q)$$

guarantees that the true likelihood:

$$\ell(\theta) = \log P_{\theta}(x)$$

will never decrease during optimization.

2. No Guarantee of Global Optimum

Although ELBO optimization improves likelihood monotonically, it does not guarantee convergence to the global optimum of:

$$\ell(\theta)$$

3. Why Use ELBO?

The true likelihood:

$$\log P_{\theta}(x)$$

is generally intractable.

Therefore we optimize the tractable lower bound:

$$J_{\theta}(q)$$

instead.

Key Takeaway

The central idea of latent variable learning is:

Replace an intractable likelihood optimization problem with optimization of a tractable lower bound (ELBO).

This principle underlies most modern probabilistic generative models.

Example Case : Gaussian Mixture Model (GMM)

In a Gaussian Mixture Model (GMM), the latent variable:

$$z$$

is discrete.

Suppose:

$$z \in \{1, 2, 3, \dots, M\}$$

where:

$$M$$

is the number of mixture components or clusters.

Latent Variable Representation

The latent variable model is:

$$P_{\theta}(x) = \sum_z P_{\theta}(x, z)$$

Using the product rule:

$$P_{\theta}(x) = \sum_{j=1}^M P_{\theta}(z = j)P_{\theta}(x|z = j)$$

Gaussian Mixture Assumption

In a GMM:

$$P_{\theta}(z = j) = \alpha_j$$

where:

$$\alpha_j$$

is the mixing coefficient of component:

$$j$$

and:

$$P_{\theta}(x|z = j) = \mathcal{N}(x; \mu_j, \Sigma_j)$$

Thus:

$$P_{\theta}(x) = \sum_{j=1}^M \alpha_j \mathcal{N}(x; \mu_j, \Sigma_j)$$

Interpretation

A GMM assumes that the data distribution is generated from a weighted combination of multiple Gaussian distributions.

Each Gaussian component corresponds to one cluster.

Parameters of a GMM

The complete parameter set is:

$$\theta = \{\alpha_1, \alpha_2, \dots, \alpha_M, \mu_1, \mu_2, \dots, \mu_M, \Sigma_1, \Sigma_2, \dots, \Sigma_M\}$$

where:

$$x \in \mathbb{R}^d$$

Thus:

$$\mu_j \in \mathbb{R}^d$$

and:

$$\Sigma_j \in \mathbb{R}^{d \times d}$$

Constraints on Mixing Coefficients

The mixing coefficients satisfy:

$$0 \leq \alpha_j \leq 1$$

and:

$$\sum_{j=1}^M \alpha_j = 1$$

Thus:

$$\alpha_j$$

represents the probability that a sample belongs to cluster:

$$j$$

Goal

Estimate:

$$\theta$$

by maximizing the ELBO:

$$\theta^*, q^* = \arg \max_{\theta, q} J_{\theta}(q)$$

Expectation Maximization (EM) Algorithm

The optimization is solved iteratively by alternating optimization over:

- $q(z|x)$
- θ

Let:

$$\theta^{(t)} \quad \text{and} \quad q^{(t)}$$

denote parameter estimates at iteration:

$$t$$

Then for:

$$t = 1, \dots, T$$

perform:

E-Step

$$q^{(t+1)} = \arg \max_q J_{\theta^{(t)}}(q)$$

with:

$$\theta^{(t)}$$

kept fixed.

M-Step

$$\theta^{(t+1)} = \arg \max_{\theta} J_{\theta}(q^{(t+1)})$$

with:

$$q^{(t+1)}$$

kept fixed.

This alternating optimization procedure is called the:

Expectation Maximization (EM) Algorithm

Likelihood Improvement Property of EM

It can be shown that EM guarantees:

$$\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$$

where:

$$\ell(\theta) = \log P_{\theta}(x)$$

Thus, every EM iteration monotonically improves the likelihood.

E-Step for GMM

For GMMs:

$$q^*(z|x) = P_{\theta}(z|x)$$

It can be analytically shown that the optimal variational distribution maximizing ELBO is the true posterior:

$$q^*(z|x) = P_{\theta}(z|x)$$

Posterior Computation Using Bayes Theorem

Using Bayes theorem:

$$P_{\theta}(z|x) = \frac{P_{\theta}(x|z)P_{\theta}(z)}{P_{\theta}(x)}$$

Substituting the GMM distributions:

$$P_{\theta}(z = j|x) = \frac{\alpha_j \mathcal{N}(x; \mu_j, \Sigma_j)}{\sum_{k=1}^M \alpha_k \mathcal{N}(x; \mu_k, \Sigma_k)}$$

This quantity is called the:

Responsibility

of component:

j

for sample:

x

Interpretation of Responsibilities

$$P_{\theta}(z = j|x)$$

represents the probability that data point:

x

belongs to cluster:

j

Thus GMM clustering is probabilistic.

A sample can partially belong to multiple clusters.

M-Step for GMM

After computing responsibilities, optimize:

$$\theta^{(t+1)} = \arg \max_{\theta} J_{\theta}(q^{(t+1)})$$

This is done by differentiating the ELBO with respect to:

$$\alpha_j, \mu_j, \Sigma_j$$

and solving for optimal parameter updates.

Closed Form EM Updates

The EM updates for GMMs have closed-form analytical solutions.

Effective Number of Points

$$N_j = \sum_{i=1}^N P_{\theta}(z = j|x_i)$$

Mixing Coefficients

$$\alpha_j = \frac{N_j}{N}$$

Means

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^N P_{\theta}(z = j|x_i)x_i$$

Covariance Matrices

$$\Sigma_j = \frac{1}{N_j} \sum_{i=1}^N P_{\theta}(z = j|x_i)(x_i - \mu_j)(x_i - \mu_j)^T$$

Relation Between K-Means and GMM

K-Means clustering is a limiting special case of GMM.

Suppose:

$$\Sigma_j \rightarrow \epsilon I$$

with:

$$\epsilon \rightarrow 0$$

Then the Gaussian distributions become extremely sharp.

As a result:

$$P_{\theta}(z = j|x)$$

approaches a one-hot assignment:

$$P_{\theta}(z = j|x) \in \{0, 1\}$$

Thus each sample belongs to exactly one cluster.

This recovers the K-Means objective.

Key Difference Between GMM and K-Means

K-Means

- Hard clustering
- Deterministic assignments
- Euclidean-distance based

GMM

- Soft clustering
- Probabilistic assignments
- Full probabilistic density model

Important Note About EM

ELBO optimization using EM is possible only if:

$$P_{\theta}(z|x)$$

can be computed analytically or efficiently.

When EM Fails

For many complex latent variable models:

$$P_{\theta}(z|x)$$

is intractable.

In such cases:

- Exact EM cannot be performed
- Posterior computation becomes impossible

This motivates approximate inference methods such as:

- Variational Inference
- Variational Autoencoders (VAE)
- Monte Carlo methods
- Diffusion-based inference

Neural Latent Variable Models

We are left with an important question:

How do we learn latent variable models when:

$$P_{\theta}(z|x)$$

is unknown or intractable, and therefore EM fails?

This motivates the development of:

Neural Latent Variable Models

Goals of Neural Latent Variable Models

The objectives are:

- Learn latent variable models even when:

$$P_{\theta}(z|x)$$

cannot be computed analytically

- Enable sampling or generation from:

$$P_{\theta}(x|z)$$

- Perform posterior inference:

$$q(z|x)$$

Latent Variable Model

Suppose:

$$P_{\theta}(x) = \int P_{\theta}(x, z) dz$$

We optimize the ELBO:

$$\theta^*, q^* = \arg \max_{\theta, q} J_{\theta}(q)$$

where:

$$J_{\theta}(q) = \mathbb{E}_{q(z|x)} \left[\log \frac{P_{\theta}(x, z)}{q(z|x)} \right]$$

Expanding the ELBO

Using:

$$P_\theta(x, z) = P_\theta(x|z)P_\theta(z)$$

we get:

$$J_\theta(q) = \mathbb{E}_{q(z|x)} \left[\log \frac{P_\theta(x|z)P_\theta(z)}{q(z|x)} \right]$$

Separating the logarithm:

$$= \mathbb{E}_{q(z|x)} [\log P_\theta(x|z)] + \mathbb{E}_{q(z|x)} \left[\log \frac{P_\theta(z)}{q(z|x)} \right]$$

Recognizing the KL divergence:

$$J_\theta(q) = \mathbb{E}_{q(z|x)} [\log P_\theta(x|z)] - D_{KL}(q(z|x) \| P_\theta(z))$$

Interpretation of ELBO Terms

Thus the ELBO contains two terms:

1. Reconstruction Term

$$\mathbb{E}_{q(z|x)} [\log P_\theta(x|z)]$$

This ensures that:

z

contains enough information to reconstruct:

x

2. Regularization Term

$$D_{KL}(q(z|x) \| P_\theta(z))$$

This forces the latent distribution to remain close to a prior distribution.
Usually:

$$P_\theta(z) = \mathcal{N}(0, I)$$

Loss Function for Neural Latent Variable Models

To optimize the ELBO using neural networks:

- Represent:

$$P_{\theta}(x|z)$$

using a neural network

- Represent:

$$q(z|x)$$

using another neural network

How Neural Networks Represent Distributions

There are two major approaches.

A. Deterministic Representation

Example:

$$z \sim \mathcal{N}(0, I)$$

$$z \rightarrow g_{\theta}(z) \rightarrow \hat{x}$$

where:

$$g_{\theta}(z)$$

is a neural network.

The model outputs samples from the distribution.

Thus:

$$g_{\theta}(z)$$

implicitly models:

$$P_{\theta}(x)$$

This is called:

Implicit Density Modeling

The model generates samples but does not explicitly compute the density function.

GANs belong to this category.

B. Probabilistic Representation

Another approach is:

$$x \rightarrow q_\phi(z|x)$$

The neural network outputs the parameters of a probability distribution.
For example:

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$$

The network outputs:

- Mean
- Variance

instead of directly generating samples.
VAEs belong to this category.

Autoencoders (AE) and Variational Autoencoders (VAE)

In Autoencoders and VAEs:

- The encoder models:

$$q_\phi(z|x)$$

- The decoder models:

$$P_\theta(x|z)$$

using neural networks.

VAE Architecture

Given:

$$D = \{x_i\}_{i=1}^N$$

with:

$$x_i \stackrel{\text{iid}}{\sim} P_X$$

and latent variable model:

$$P_\theta(x) = \int P_\theta(x, z) dz$$

the VAE optimizes:

$$J_\theta(q) = \mathbb{E}_{q(z|x)}[\log P_\theta(x|z)] - D_{KL}(q(z|x) \parallel P_\theta(z))$$

Encoder-Decoder Structure

$$x \rightarrow q_\phi(z|x)$$

The encoder outputs:

$$\mu_\phi(x), \sigma_\phi(x)$$

which parameterize:

$$q_\phi(z|x)$$

Then using the reparameterization trick:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

Next:

$$z \rightarrow P_\theta(x|z)$$

The decoder reconstructs the data.

Important Note

There is no direct connection between encoder and decoder parameters.

$$\phi$$

and:

$$\theta$$

represent independent neural network parameters.

Optimization

The optimization problem becomes:

$$\phi^*, \theta^* = \arg \max_{\phi, \theta} J_\theta(q_\phi)$$

Gradient ascent updates:

$$\phi^{(t+1)} = \phi^{(t)} + \alpha \nabla_\phi J_\theta(q_\phi)$$

$$\theta^{(t+1)} = \theta^{(t)} + \alpha \nabla_\theta J_\theta(q_\phi)$$

Autoencoder (AE) as a Special Case of VAE

A standard autoencoder can be viewed as a limiting special case of a VAE.

Autoencoder Objective

A deterministic autoencoder optimizes only reconstruction:

$$J_{AE} = \mathbb{E}[\log P_{\theta}(x|z)]$$

There is no probabilistic latent regularization.

Thus:

$$D_{KL}(q(z|x) \parallel P(z))$$

is absent.

VAE Objective

VAE optimizes:

$$J_{VAE} = \mathbb{E}_{q(z|x)}[\log P_{\theta}(x|z)] - D_{KL}(q(z|x) \parallel P(z))$$

Limiting Case

Suppose:

$$q(z|x)$$

collapses to a deterministic point:

$$q(z|x) = \delta(z - f_{\phi}(x))$$

where:

$$\delta$$

is the Dirac delta distribution.

Then:

$$z = f_{\phi}(x)$$

becomes deterministic.

In this limit:

- Sampling disappears
- Variance becomes zero
- KL regularization vanishes

Thus the VAE reduces to a standard autoencoder.

Key Difference Between AE and VAE

Autoencoder

- Deterministic latent representation
- Only reconstruction objective
- No probabilistic sampling

Variational Autoencoder

- Probabilistic latent representation
- Includes KL regularization
- Enables sampling and generation

Key Takeaway

Neural latent variable models use neural networks to:

- Learn latent representations
- Approximate posterior inference
- Model complex probability distributions
- Enable generative modeling

The VAE framework combines:

- Variational inference
- Probabilistic modeling
- Deep neural networks

into a scalable latent variable learning framework.

Training VAE : Reparameterization Trick

Data Flow in Autoencoder (AE)

In a standard autoencoder:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

or more explicitly:

$$x \rightarrow q_\phi(z|x) \rightarrow z \rightarrow P_\theta(x|z) \rightarrow \hat{x}$$

The latent representation:

$$z$$

is deterministic.

Thus there exists a direct differentiable connection between:

- Encoder
- Latent representation
- Decoder

Backpropagation can be performed directly.

Data Flow in Variational Autoencoder (VAE)

In a VAE:

$$x \rightarrow q_\phi(z|x) \rightarrow (\mu_\phi(x), \sigma_\phi(x)) \rightarrow z \rightarrow P_\theta(x|z) \rightarrow \hat{x}$$

The encoder outputs parameters of a probability distribution:

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$$

Then:

$$z$$

is sampled from this distribution.

Important Difference

In VAE:

- Sampling operation exists
- Sampling is stochastic
- Encoder and decoder are not directly connected

The parameters:

$$\phi$$

and:

$$\theta$$

denote encoder and decoder weights respectively.

VAE Objective

The ELBO objective optimized in VAE is:

$$J_\theta(q) = \mathbb{E}_{q_\phi(z|x)}[\log P_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel P_\theta(z))$$

Problem : Computing Gradients Through Sampling

Consider the reconstruction term:

$$\mathbb{E}_{q_\phi(z|x)}[\log P_\theta(x|z)]$$

We want to compute gradients with respect to:

$$\phi$$

Define:

$$P_\psi(v) = q_\phi(z|x)$$

and:

$$f_\psi(v) = \log P_\theta(x|z)$$

where:

$$v \rightarrow z$$

and:

$$\psi \rightarrow \phi$$

Then:

$$\nabla_\psi \mathbb{E}_{P_\psi(v)}[f_\psi(v)]$$

Expanding expectation:

$$= \nabla_\psi \int P_\psi(v) f_\psi(v) dv$$

Moving gradient inside integral:

$$= \int \nabla_\psi (P_\psi(v) f_\psi(v)) dv$$

Using product rule:

$$= \int (\nabla_\psi f_\psi(v)) P_\psi(v) dv + \int (\nabla_\psi P_\psi(v)) f_\psi(v) dv$$

Thus:

$$= \mathbb{E}_{P_\psi(v)}[\nabla_\psi f_\psi(v)] + \text{additional term}$$

Problem

The second term:

$$\int (\nabla_\psi P_\psi(v)) f_\psi(v) dv$$

is not an expectation.

Hence it cannot be efficiently estimated using Monte Carlo sampling.

Therefore:

$$\nabla_\phi \mathbb{E}_{q_\phi(z|x)}[\log P_\theta(x|z)]$$

cannot be directly computed.

Important Conclusion

Sampling operation is not directly differentiable.

This prevents standard backpropagation through stochastic latent variables.

Solution : Reparameterization Trick

The solution is the:

Reparameterization Trick

This is a classical statistical technique where one random variable is represented as a deterministic transformation of another random variable.

Core Idea

Suppose:

$$v \sim P_\psi(v)$$

Instead of sampling directly from:

$$P_\psi(v)$$

introduce another random variable:

$$\epsilon \sim P_\epsilon$$

such that:

$$P_\epsilon$$

does not depend on:

$$\psi$$

Now define:

$$v = g_\psi(\epsilon)$$

Then:

$$\mathbb{E}_{P_\psi(v)}[f_\psi(v)] = \mathbb{E}_{P_\epsilon}[f_\psi(g_\psi(\epsilon))]$$

This follows from the:

Law of the Unconscious Statistician (LOTUS)

Key Advantage

Now:

$$\nabla_\psi \mathbb{E}_{P_\psi(v)}[f_\psi(v)] = \nabla_\psi \mathbb{E}_{P_\epsilon}[f_\psi(g_\psi(\epsilon))]$$

The sampling distribution:

$$P_\epsilon$$

is independent of:

$$\psi$$

Thus gradients can move through:

$$g_\psi(\epsilon)$$

using standard backpropagation.

Monte Carlo Approximation

Using:

$$M$$

samples:

$$\nabla_\psi \mathbb{E}_{P_\epsilon}[f_\psi(g_\psi(\epsilon))] \approx \frac{1}{M} \sum_{j=1}^M \nabla_\psi f_\psi(g_\psi(\epsilon_j))$$

where:

$$\epsilon_j \sim P_\epsilon$$

Now the gradient is computable.

Gaussian Reparameterization

Suppose:

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$$

The encoder outputs:

$$\mu_\phi(x), \quad \sigma_\phi(x)$$

Introduce:

$$\epsilon \sim \mathcal{N}(0, I)$$

Then define:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

where:

$$\odot$$

denotes element-wise multiplication.

Thus:

$$z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$$

Interpretation

Instead of directly sampling:

$$z$$

we:

- Sample simple noise:

$$\epsilon \sim \mathcal{N}(0, I)$$

- Transform it deterministically

Thus stochasticity is separated from trainable parameters.

Affine Transformation Interpretation

The mapping:

$$z = \mu_\phi(x) + \sigma_\phi(x)\epsilon$$

is an affine transformation of Gaussian noise.

Important Observation

Any Gaussian distribution can be generated from a standard normal distribution using affine transformation.

Thus:

$$\mathcal{N}(\mu, \sigma) = \mu + \sigma\epsilon$$

with:

$$\epsilon \sim \mathcal{N}(0, I)$$

Inverse CDF Reparameterization

Another general method is the inverse CDF method.

Suppose:

$$Z$$

is a random variable with cumulative distribution function:

$$F_Z(z)$$

Sample:

$$\epsilon \sim U[0, 1]$$

Then define:

$$z = F_Z^{-1}(\epsilon)$$

Thus:

$$z \sim P_Z$$

This transforms uniform random variables into arbitrary distributions.

Examples

Uniform to Exponential

If:

$$z \sim \text{Exponential}(\lambda)$$

then:

$$F_Z(z) = 1 - e^{-\lambda z}$$

Thus:

$$F_Z^{-1}(\epsilon) = -\frac{1}{\lambda} \log(1 - \epsilon)$$

where:

$$\epsilon \sim U[0, 1]$$

Why Reparameterization is Important

The reparameterization trick enables:

- Differentiable sampling
- Backpropagation through stochastic variables
- Efficient optimization of ELBO
- Scalable neural variational inference

Without it, gradient-based optimization of VAEs would not be feasible.

Key Takeaway

The reparameterization trick transforms stochastic sampling into a deterministic differentiable transformation of noise.

This allows:

gradient descent through probabilistic latent variables

and is the foundational idea enabling efficient training of VAEs.

Final Training of Variational Autoencoder (VAE)

Forward Computational Graph

Given a data sample:

$$x_i \in \mathbb{R}^d$$

the encoder network computes:

$$q_\phi(z|x_i)$$

which outputs:

$$\mu_\phi(x_i) \quad \text{and} \quad \Sigma_\phi(x_i)$$

where:

$$z_i \in \mathbb{R}^k$$

$$\mu_\phi(x_i) \in \mathbb{R}^k$$

$$\Sigma_\phi(x_i) \in \mathbb{R}^{k \times k}$$

Typically:

$$\Sigma_\phi(x_i)$$

is assumed diagonal.

Sampling via Reparameterization

Sample:

$$\epsilon_i \sim \mathcal{N}(0, I)$$

Then compute:

$$z_i = \mu_\phi(x_i) + \Sigma_\phi(x_i)\epsilon_i$$

Thus:

$$z_i \sim q_\phi(z|x_i) = \mathcal{N}(\mu_\phi(x_i), \Sigma_\phi(x_i))$$

This is obtained using the reparameterization trick.

Decoder

The latent sample:

$$z_i$$

is passed through the decoder:

$$z_i \rightarrow P_\theta(x|z_i) \rightarrow \hat{x}$$

The decoder reconstructs the data.

Reconstruction Term in ELBO

Recall the ELBO:

$$J_\theta(q_\phi) = \mathbb{E}_{q_\phi(z|x)}[\log P_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel P(z))$$

We first focus on:

$$\mathbb{E}_{q_\phi(z|x)}[\log P_\theta(x|z)]$$

Using reparameterization:

$$= \mathbb{E}_{P_\epsilon}[\log P_\theta(x|g_\phi(\epsilon))]$$

Monte Carlo approximation:

$$\approx \frac{1}{M} \sum_{j=1}^M \log P_\theta(x|g_\phi(\epsilon_j))$$

where:

$$\epsilon_j \sim \mathcal{N}(0, I)$$

Thus:

$$\nabla_\phi \mathbb{E}_{q_\phi(z|x)}[\log P_\theta(x|z)]$$

becomes computable.

Likelihood Modeling

To compute:

$$\log P_\theta(x|z)$$

we assume a probabilistic decoder model.

Example:

$$P_\theta(x|z) = \mathcal{N}(x; \hat{x}_\theta(z), I)$$

where:

$$\hat{x}_\theta(z)$$

is the decoder output.

Gaussian Likelihood

For Gaussian decoder:

$$P_{\theta}(x|z) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{1}{2}\|x - \hat{x}_{\theta}(z)\|^2\right)$$

Taking logarithm:

$$\log P_{\theta}(x|z) = -\frac{1}{2}\|x - \hat{x}_{\theta}(z)\|^2 + C$$

where:

$$C$$

is a constant independent of parameters.

Thus maximizing likelihood is equivalent to minimizing reconstruction error.

Reconstruction Loss

Therefore:

$$\mathbb{E}_{q_{\phi}(z|x)}[\log P_{\theta}(x|z)] \approx -\frac{1}{M} \sum_{j=1}^M \|x_i - \hat{x}_{\theta}(z_j)\|^2$$

where:

$$z_j = \mu_{\phi}(x_i) + \Sigma_{\phi}(x_i)\epsilon_j$$

and:

$$\epsilon_j \sim \mathcal{N}(0, I)$$

Forward Pass of VAE

Step 1 : Encoder

Given:

$$x_i$$

pass through encoder:

$$x_i \rightarrow q_{\phi}(z|x_i)$$

to obtain:

$$\mu_{\phi}(x_i), \Sigma_{\phi}(x_i)$$

Step 2 : Sample Noise

Sample:

$$\epsilon_1, \epsilon_2, \dots, \epsilon_M \sim \mathcal{N}(0, I)$$

outside the network.

Step 3 : Reparameterization

Compute latent vectors:

$$z_j = \mu_\phi(x_i) + \Sigma_\phi(x_i)\epsilon_j$$

Step 4 : Decoder

Pass latent vectors through decoder:

$$z_j \rightarrow \hat{x}_\theta(z_j)$$

Step 5 : Reconstruction Loss

Compute:

$$-\frac{1}{M} \sum_{j=1}^M \|x_i - \hat{x}_\theta(z_j)\|^2$$

Backward Pass

Compute gradients:

$$\nabla_\phi \left[-\frac{1}{M} \sum_{j=1}^M \|x_i - \hat{x}_\theta(z_j)\|^2 \right]$$

Backpropagation occurs through:

- Decoder
- Reparameterization operation
- Encoder

Thus we obtain gradients for the first ELBO term.

Second Term in ELBO : KL Divergence

The second ELBO term is:

$$D_{KL}(q_\phi(z|x) \parallel P(z))$$

Usually:

$$P(z) = \mathcal{N}(0, I)$$

and:

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \Sigma_\phi(x))$$

Closed Form KL Divergence

For Gaussian distributions:

$$D_{KL}(\mathcal{N}(\mu, \Sigma) \parallel \mathcal{N}(0, I)) = \frac{1}{2} (\text{tr}(\Sigma) + \mu^T \mu - k - \log \det(\Sigma))$$

where:

$$k$$

is latent dimension.

Expanding:

$$= -\frac{1}{2} \log \det(\Sigma_\phi(x)) - k + \text{tr}(\Sigma_\phi(x)) + \|\mu_\phi(x)\|^2$$

All quantities are computable scalars.

Total Gradient for Encoder

Thus:

$$\begin{aligned} \nabla_\phi J_\theta(q_\phi) &= \nabla_\phi \left[-\frac{1}{M} \sum_{j=1}^M \|x_i - \hat{x}_\theta(z_j)\|^2 \right] \\ &+ \nabla_\phi \left[-\frac{1}{2} \log \det(\Sigma_\phi(x)) - k + \text{tr}(\Sigma_\phi(x)) + \|\mu_\phi(x)\|^2 \right] \end{aligned}$$

Encoder update:

$$\phi^{(t+1)} = \phi^{(t)} + \alpha \nabla_\phi J_\theta(q_\phi)$$

This constitutes one training step for the encoder.

Training the Decoder

Now consider gradients with respect to:

$$\theta$$

$$\nabla_{\theta} J_{\theta}(q_{\phi}) = \nabla_{\theta} \mathbb{E}_{q_{\phi}(z|x)} [\log P_{\theta}(x|z)]$$

because:

$$\nabla_{\theta} D_{KL}(q_{\phi}(z|x) \| P(z)) = 0$$

The KL divergence does not depend on:

$$\theta$$

Thus decoder optimization involves only reconstruction likelihood.

Decoder update:

$$\theta^{(t+1)} = \theta^{(t)} + \alpha \nabla_{\theta} J_{\theta}(q_{\phi})$$

Summary of VAE Training

Encoder Learns

- Mean:

$$\mu_{\phi}(x)$$

- Variance:

$$\Sigma_{\phi}(x)$$

- Approximate posterior:

$$q_{\phi}(z|x)$$

Decoder Learns

- Conditional likelihood:

$$P_{\theta}(x|z)$$

- Reconstruction mapping:

$$z \rightarrow x$$

Final VAE Loss

The complete VAE objective becomes:

$$\mathcal{L}_{VAE} = \underbrace{\frac{1}{M} \sum_{j=1}^M \|x - \hat{x}_{\theta}(z_j)\|^2}_{\text{Reconstruction Loss}} + \underbrace{D_{KL}(q_{\phi}(z|x) \| P(z))}_{\text{Latent Regularization}}$$

Key Takeaway

VAE training combines:

- Probabilistic latent modeling
- Variational inference
- Monte Carlo estimation
- Reparameterized differentiable sampling
- Neural network optimization

into a scalable generative learning framework.

Inference with Variational Autoencoders (VAE)

Once the VAE is trained, it can be used for two major purposes:

- Data generation (sampling)
- Latent posterior inference / embedding / feature extraction

(i) Data Generation or Sampling

Recall the VAE model:

$$P_{\theta}(x) = \int P_{\theta}(x|z)P(z) dz$$

where:

$$P(z) = \mathcal{N}(0, I)$$

After training, the decoder:

$$P_{\theta^*}(x|z)$$

acts as a generative model.

Sampling Procedure

To generate a new data sample:

Step 1 : Sample Latent Variable

Sample:

$$z_{\text{new}} \sim P(z) = \mathcal{N}(0, I)$$

Step 2 : Pass Through Decoder

Generate:

$$x_{\text{new}} \sim P_{\theta^*}(x|z_{\text{new}})$$

using the trained decoder.

Decoder Output

The decoder outputs the parameters of:

$$P_{\theta}(x|z)$$

For example:

$$P_{\theta}(x|z) = \mathcal{N}(\hat{x}_{\theta}(z), I)$$

where:

$$\hat{x}_{\theta}(z)$$

is the mean of the conditional distribution.

Thus:

$$z_{\text{new}} \rightarrow P_{\theta^*}(x|z_{\text{new}}) \rightarrow \hat{x}_{\theta^*}(z_{\text{new}})$$

Why Sampling from $\mathcal{N}(0, I)$ Works

Recall the ELBO:

$$J_{\theta}(q_{\phi}) = \mathbb{E}_{q_{\phi}(z|x)}[\log P_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \parallel P(z))$$

The KL divergence term forces:

$$q_{\phi}(z|x) \approx P(z) = \mathcal{N}(0, I)$$

for all:

$$x$$

Thus the latent space learned by the encoder approximately follows a standard normal distribution.

Therefore:

$$z \sim \mathcal{N}(0, I)$$

corresponds to sampling latent vectors from regions where the encoder has mapped the training data.

Important Interpretation

If the VAE is well trained:

$$q_{\phi}(z|x) \approx \mathcal{N}(0, I)$$

Hence:

Sampling from $\mathcal{N}(0, I)$

approximately corresponds to:

sampling latent representations of real data

Two Ways to Generate Data

A. Deterministic Generation

Use the decoder mean directly:

$$x_{\text{new}} = \hat{x}_{\theta^*}(z_{\text{new}})$$

This is the most common approach.

B. Probabilistic Generation

Sample from the decoder distribution:

$$x_{\text{new}} \sim \mathcal{N}(\hat{x}_{\theta^*}(z_{\text{new}}), I)$$

This introduces stochasticity into generated samples.

Important Note

During generation:

- Encoder is not used
- Only decoder is required

Generation pipeline:

$$z_{\text{new}} \sim \mathcal{N}(0, I) \rightarrow \text{Decoder} \rightarrow x_{\text{new}}$$

Latent Space Interpretation

The latent space learned by the VAE is:

- Continuous
- Smooth
- Structured

Nearby latent vectors generate similar outputs.

Thus interpolation in latent space produces meaningful semantic transformations.

(ii) VAE for Posterior Inference / Embedding / Feature Extraction

The encoder of a trained VAE can be used as a feature extractor.

Encoder Inference

Given input:

$$x$$

the encoder computes:

$$q_{\phi}(z|x) = \mathcal{N}(\mu_{\phi}(x), \Sigma_{\phi}(x))$$

Pipeline:

$$x \rightarrow q_{\phi}(z|x) \rightarrow \mu_{\phi}(x), \Sigma_{\phi}(x)$$

Latent Representation

The feature vector or embedding for:

$$x$$

can be defined in two ways.

A. Deterministic Embedding

Use the mean:

$$z = \mu_{\phi}(x)$$

This is the most common embedding representation.

B. Probabilistic Embedding

Sample:

$$z \sim \mathcal{N}(\mu_\phi(x), \Sigma_\phi(x))$$

This retains uncertainty information.

Dimensionality Reduction

Typically:

$$z \in \mathbb{R}^k$$

where:

$$k \ll d$$

Thus the encoder performs dimensionality reduction.

Example:

$$x \in \mathbb{R}^{784}$$

(MNIST image)
may be mapped to:

$$z \in \mathbb{R}^2$$

or:

$$z \in \mathbb{R}^{32}$$

Applications of VAE Embeddings

- Dimensionality reduction
- Clustering
- Visualization
- Representation learning
- Semantic interpolation
- Anomaly detection
- Feature extraction

Latent Space Geometry

The KL regularization term ensures:

- Smooth latent space
- Continuous representations
- Structured embeddings

This is a major advantage over standard autoencoders.

Comparison : AE vs VAE Latent Space

Autoencoder

- Arbitrary latent representations
- Disconnected latent regions possible
- Poor generative capability

Variational Autoencoder

- Latent vectors follow known prior
- Smooth continuous latent space
- Good generative capability

Key Takeaway

A trained VAE provides:

- A generative model:

$$z \rightarrow x$$

- An inference model:

$$x \rightarrow z$$

- A low-dimensional latent representation
- A probabilistically regularized latent space

Thus VAEs simultaneously perform:

generation + inference + representation learning

β -Variational Autoencoder (β -VAE)

Posterior Collapse in Naive VAE

Recall the ELBO objective of a standard VAE:

$$J_{\theta}(q_{\phi}) = \mathbb{E}_{q_{\phi}(z|x)}[\log P_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \parallel P(z))$$

Equivalently, using reconstruction loss:

$$J_{\theta}(q_{\phi}) = \|x - \hat{x}_{\theta}(z)\|^2 + D_{KL}(q_{\phi}(z|x) \parallel P(z))$$

where:

- First term:

$$\|x - \hat{x}_{\theta}(z)\|^2$$

is the reconstruction term

- Second term:

$$D_{KL}(q_{\phi}(z|x) \parallel P(z))$$

is the latent regularization term

What Does the KL Term Do?

The KL divergence term forces:

$$q_{\phi}(z|x) \approx P(z) = \mathcal{N}(0, I)$$

for all:

$$x$$

Thus all latent posteriors are pushed toward the same distribution.

Posterior Collapse

Suppose:

$$x_i, x_j$$

are two different input samples.

The KL term may force:

$$q_{\phi}(z|x_i) = q_{\phi}(z|x_j) = \mathcal{N}(0, I)$$

Thus:

$$z_i \sim q_{\phi}(z|x_i)$$

and:

$$z_j \sim q_\phi(z|x_j)$$

become statistically very similar.

Problem

The decoder receives almost identical latent vectors for different inputs.

Therefore the decoder struggles to distinguish between:

$$x_i \quad \text{and} \quad x_j$$

This weakens reconstruction quality.

Definition : Posterior Collapse

When the latent posterior distributions collapse toward the prior distribution for all inputs,

$$q_\phi(z|x) \approx P(z)$$

the latent variables stop carrying meaningful information about the data.

This phenomenon is called:

Posterior Collapse

Effect of Posterior Collapse

- Latent space loses information
- Decoder ignores latent variables
- Reconstruction quality degrades
- Model behaves like unconditional generator

Understanding VAE as a Regularized Autoencoder

Recall the VAE loss:

$$\|x - \hat{x}_\theta(z)\|^2 + D_{KL}(q_\phi(z|x) \parallel P(z))$$

This can be interpreted as:

Reconstruction Cost + Regularization Cost

Thus:

VAE = Regularized Autoencoder

What is Regularization?

Regularization refers to constraining the parameters or activations of a model so that certain desirable behavior is imposed.

Typical regularized optimization:

$$L(\theta) + \lambda\Omega(\theta)$$

where:

- $L(\theta)$
is the primary cost function
- $\Omega(\theta)$
is the regularization function
- λ
controls regularization strength

Effect of Regularization Constant

Very Small λ

- Weak regularization
- Model may overfit

Very Large λ

- Strong regularization
- Model may underfit

Observation About Naive VAE

In the standard VAE objective:

$$\|x - \hat{x}_\theta(z)\|^2 + D_{KL}(q_\phi(z|x) \parallel P(z))$$

there is no explicit regularization coefficient.

The KL term has fixed importance.

This may excessively regularize the latent space and cause posterior collapse.

Solution : β -VAE

Introduce a weighting coefficient:

$$\beta$$

to control KL regularization strength.

The new objective becomes:

$$\hat{J}_\theta(q_\phi) = \|x - \hat{x}_\theta(z)\|^2 + \beta D_{KL}(q_\phi(z|x) \parallel P(z))$$

This model is called:

β -Variational Autoencoder (β -VAE)

Interpretation of β

$$\beta$$

controls the trade-off between:

- Reconstruction quality
- Latent regularization

Small β

If:

$$\beta \rightarrow 0$$

then:

- KL regularization becomes weak
- Model behaves similar to autoencoder
- Reconstruction quality improves

- Latent space may become irregular

However:

$$q_{\phi}(z|x) \not\approx P(z)$$

Thus generation quality may degrade.

Large β

If:

$$\beta$$

is large:

- Strong latent regularization
- Better latent structure
- Better generative capability

However:

- Reconstruction quality decreases
- Posterior collapse may occur

Trade-off in β -VAE

Low β

Better reconstruction

but:

Poor generation

High β

Better generation

but:

Poor reconstruction

Key Insight

The KL term tries to make latent representations simple.
The reconstruction term tries to preserve information.
These two objectives compete with each other.

$$\beta$$

controls this competition.

Latent Space Effect

Increasing:

$$\beta$$

typically leads to:

- More disentangled latent representations
- Smoother latent space
- Better interpolation

but weaker reconstructions.

Disentangled Representation Learning

One major application of β -VAE is learning disentangled latent representations.
Different latent dimensions may independently represent:

- Rotation
- Object size
- Illumination
- Shape
- Texture

This emerges due to stronger latent regularization.

Final Objective of β -VAE

$$\mathcal{L}_{\beta\text{-VAE}} = \|x - \hat{x}_{\theta}(z)\|^2 + \beta D_{KL}(q_{\phi}(z|x) \| P(z))$$

Key Takeaway

β -VAE generalizes standard VAE by introducing a tunable regularization parameter:

$$\beta$$

which controls the balance between:

reconstruction fidelity

and:

latent space regularization

Thus:

$$\beta\text{-VAE} = \text{Regularized Probabilistic Autoencoder}$$